

PLATAFORMAS BIG DATA

3º de Ingeniería Informática

Práctica 2

El acceso a los datos

Contexto

Una vez aprendidos los principales formatos de almacenamiento de matrices en memoria, es hora de abordar un caso real de uso. Como se ha comentado en la práctica anterior, la IA se aplica actualmente en multitud de campos: desarrollo de software, agricultura, física, química, biología, empresas privadas de diversos sectores, etc. Además, junto con la Computación de Altas Prestaciones (HPC) y el procesamiento del Big Data ha permitido entender y obtener información de millones y millones de datos que hasta hace poco tiempo carecían de sentido para la humanidad (procesamiento del Big Data).

Por todos estos motivos, es imprescindible conocer, tanto a nivel alto como a nivel bajo, el funcionamiento de las redes neuronales.

Objetivos

En esta práctica el alumnado deberá implementar una red neuronal sencilla, desde cero, en lenguaje de programación C. Esto ayudará al alumnado a comprender las operaciones de álgebra lineal básicas que se llevan a cabo en el interior de una red neuronal, así como la complejidad computacional de las mismas.

Dicho esto, los objetivos principales de la práctica son:

- Comprensión del alumnado del funcionamiento de una red neuronal simple.
- Llevar a cabo de manera adecuada el acceso a los datos de entrada de una red neuronal.
- Conocimiento y comprensión de las funciones básicas que conforman una red neuronal simple.
- Manejar con soltura el acceso a los datos de una matriz almacenada en memoria serializada, tanto en formato row-major como en column-major.

EJERCICIO 1

La red neuronal más sencilla para implementar desde cero en C es un **Perceptrón Multicapa (MLP)** de una sola capa oculta. Las redes neuronales MLP se caracterizan por la siguiente arquitectura:

- **Capa de entrada:** Corresponde directamente al número de características o variables del problema.
- **Capa oculta:** Una única capa intermedia (3 - 5 neuronas) para aprender relaciones no lineales.
- **Capa de salida:** El resultado final de la red.

El funcionamiento de cualquier red neuronal es como el montaje en cadena de un producto. La red neuronal recibe como entrada los datos para ir pasando por diferentes etapas hasta generar una salida.

Nuestro MLP va a estar formado por dos funciones principales:

1. Suma ponderada (paso lineal). Esta rutina llevará a cabo el producto escalar entre las entradas y los pesos más el sesgo.

$$z = \left(\sum_{i=1}^n w_i \cdot x_i \right) + b$$

2. Función de activación (ReLU). Permite que la red aprenda relaciones complejas “rompiendo” la linealidad (convertirá valores negativos en cero).

$$f(z) = \max(0, z)$$

- **xi:** Son las entrada que vienen de la capa anterior
- **wi:** Son los pesos almacenados provenientes del entrenamiento
- **b:** Es el bias, el umbral de activación de la neurona
- **z:** Es el resultado de la inferencia antes de ser filtrado por la activación

Este primer ejercicio consistirá en que el alumno lleve a cabo las reservas correspondientes de memoria. Para ello, necesitaremos definir en 3 variables los tamaños principales de nuestras entradas y salidas:

- **BATCH**: Es el número de muestras por IN_DIM.
- **IN_DIM**: Es el número de valores por muestra que le entra a nuestra red neuronal
 - En caso de una red de sensores, el valor de cada sensor.
 - En el caso de una imagen, los píxeles de una imagen.
 - etc.
- **OUT_DIM**: Número de neuronas de la capa actual

Para empezar, fijaremos:

- **BATCH=5**
- **IN_DIM=4096**
- **OUT_DIM=2048**

Seguidamente, el alumno llevará a cabo la reserva de los 3 buffers necesarios, A, B y C. Donde **A** será la entrada, de tamaño **A=BATCH * IN_DIM**, **B** los pesos de tamaño **B=IN_DIM * OUT_DIM**, y **C** la salida de tamaño **C=BATCH * OUT_DIM**. El tipo de datos de los buffers será de tipo flotante de simple precisión (*Nota: cuidado con el desbordamiento de los tamaños del buffer*).

Una vez hecha la reserva de los buffers, inicializamos cada una de las posiciones con valores. En este caso tendríamos las lecturas de los sensores, los pesos de las capas, etc. Como es una simulación, inicializamos los valores de entrada y salida a 0.01f, y los valores de los pesos a 0.02f.

EJERCICIO 2

En el siguiente paso, se implementará la capa forward, que corresponde a la suma ponderada. Siguiendo la fórmula proporcionada en el ejercicio anterior, el alumno llevará a cabo su implementación. Para ello, debe tener presente que la matriz de entrada A se considera que está almacenada en formato *row-major*. Esto implica que las muestras de un mismo sensor o los píxeles de una misma imagen se almacenan de forma consecutiva, o lo que es lo mismo, formarán la fila de una matriz. Por consiguiente, el número de filas vendrá dado por el tamaño de batch (*Nota: No olvide sumar al producto de cada entrada el bias correspondiente: $C = (A \times B) + Bias$ según la definición:*

$$z = \left(\sum_{i=1}^n w_i \cdot x_i \right) + b$$

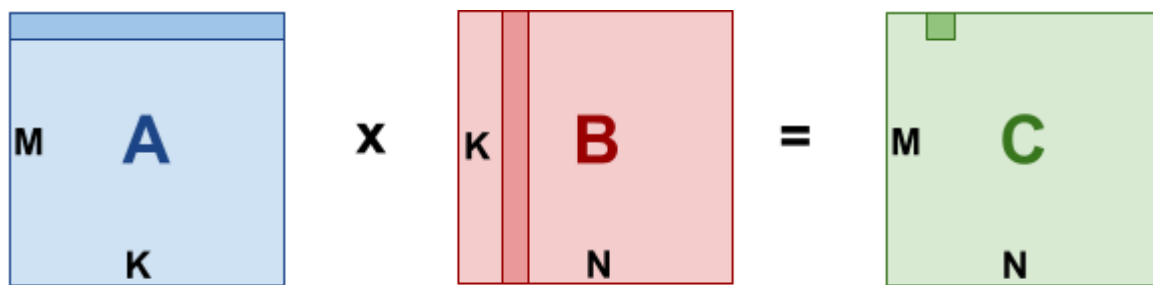


Figura 1. Esquema del producto matricial GEMM.

EJERCICIO 3

Una vez implementada la capa lineal, el alumno implementará la capa ReLU. Esta capa recibirá la salida de la capa anterior, es decir, la matriz C, y aplicará la función por definición:

$$f(z) = \max(0, z)$$

EJERCICIO 4

Como último ejercicio, se temporizará cada una de las dos capas que forman parte de nuestro MLP por separado y se llevará a cabo un estudio de cómo afecta el tamaño de batch al tiempo de procesamiento. Montar una gráfica donde el eje Y sea el tiempo de procesamiento y el eje X el tamaño de batch. La gráfica debe mostrar cómo crece el tiempo de la capa lineal en función del tamaño de batch.

Nota: Es importante que el alumno lleve a cabo los estudios de rendimiento en la misma plataforma durante el transcurso de todas las prácticas de la asignatura.